

MBS CRM — Rich Text Editor: Verification Report

Generated Jul 12, 2026 · Report only — no changes made · Confidential

1 Deployment ⚠ PARTIAL

Live deployment last started at **Jul 12, 2026 23:14 UTC** (log timestamp `1783898079435`), corresponding to commit **9c6dcda** ("Published your App").

The core RTE implementation is **live** — it was included in 9c6dcda. Four subsequent commits are not yet deployed:

COMMIT	DESCRIPTION
f46464c	Dialog polish for rich text editor (spacing, responsiveness)
ef54fa7	Weekly timeline PDF update
f24b84b	Timeline PDF added
904a187	Checkpoint commit

Current GitHub HEAD: **904a187**. Not republishing per task instructions.

2 Rich Text Editor — Where Applied ✓ PASS

email-templates.tsx — RichTextEditor wired at line 148. The only remaining Textarea reference is `dangerouslySetInnerHTML` at line 204, which is the read-only preview pane — not an editor input. Old "Body (HTML)" raw textarea is gone.

lead-detail.tsx — RichTextEditor wired at line 1074 (email body). Four remaining Textarea instances are intentional: notes field (line 343), SMS body (line 964 — plaintext correct for SMS), and two AI draft instruction popovers (lines 945, 1049 — one-line prompt fields, not outgoing content).

flyer-templates.tsx — RichTextEditor at line 273 (visual mode). Textarea at line 281 is the HTML source half of the Visual/HTML toggle — intentional and correct.

3 Variable Insertion at Cursor ✓ PASS

Logic in `components/ui/rich-text-editor.tsx` , line 84:

```
editor.chain().focus().insertContent(v).run();
```

`focus()` restores the cursor to its last editor position; `insertContent(v)` injects the token there. Chips render as `onClick` buttons. Tokens insert at cursor — not appended to end, not requiring manual typing.

4 Flyer PDF Security ✓ PASS

4a — `escapeHtml` applied to values

`renderPdf.ts` lines 54–66 confirm escaping of `&` `<` `>` `"` `'` for every substituted value. Template HTML is left intact; only values are escaped:

```
function escapeHtml(value: string): string {
  return value
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#39;");
}

export function renderTemplate(htmlTemplate, fieldValues) {
  return htmlTemplate.replace(/{{(w+)}}/g, (_, key)) => {
    const raw = fieldValues[key];
    return raw == null ? "" : escapeHtml(String(raw)); // called on every value
  });
}
```

4b — Code path from editor to PDF

Editor emits HTML → saved to `flyer_templates.html_template` in DB → API route calls

`renderTemplate(template.htmlTemplate, fieldValues)` → escaped HTML passed to `renderPdf(html)` → Puppeteer `page.setContent()` → `page.pdf()` → Buffer returned. Chain is unbroken.

4c — Injection with < or " in a field value

A value such as `<script>alert(1)</script>` is transformed by `escapeHtml` to

`<script>alert(1)</script>` before insertion. Puppeteer receives it as literal text content, not executable markup. PDF renders the escaped string visually — no injection possible.

5 Stored Format & Seeded Templates ✓ PASS

Stored format is HTML — unchanged. `RichTextEditor` receives the `value` prop as an HTML string, Tiptap parses it into its document model on mount, and `onUpdate` emits HTML back via `editor.getHTML()`. The 7 seeded starter templates use standard HTML (`<p>`, `<strong>`, `<ul>`, `{{variable}}` tokens) — all handled natively by Tiptap's StarterKit. No corruption path exists.

6 Flyer Visual / HTML-Source Toggle ✓ PASS

`flyer-templates.tsx` lines 211–287 confirm a working toggle. Label text: *"Visual editor" / "HTML source — full document control..."*. Both branches read and write the same `form.htmlTemplate` field, so switching modes preserves content.

7 Build (tsc --build) ✓ PASS

`tsc --build` exits **0** on the current committed code. No type errors.

8 Anything Incomplete

Dialog polish not live: Commit `f46464c` contains additional modal spacing and responsiveness polish that has not been deployed (4 commits behind). Core RTE functionality is live.

SMS composer is a plain Textarea: Intentional — SMS messages are plaintext, not HTML.

AI draft instruction fields are plain Textareas: Intentional — these are one-line prompt fields for the AI model, not outgoing content.

Everything else is fully implemented. No skipped items or workarounds in the RTE itself.

SUMMARY

1. Deployment	⚠ Partial — core live, 4 commits undeployed
2. RTE applied in all three locations	✓ Pass
3. Variable chips insert at cursor	✓ Pass
4. Flyer PDF security (all sub-checks)	✓ Pass
5. Stored HTML unchanged; seeded templates render	✓ Pass
6. Flyer Visual / HTML toggle	✓ Pass
7. tsc --build exits 0	✓ Pass